CSC3310 Program #2 – Divide and Conquer

Leo Borrelli, Josiah Reinholz, Trent Weiler

Approach

To find the k-th smallest number, we used a divide and conquer method. First, we pick a random number to be our pivot, then iterate through the array dividing into sub arrays smaller, equal, and greater than the pivot. We can then use the size of the arrays to determine which sub array K is in. After determining we recursively call our method, passing in the correct subarray. If K is in the greater sub array we subtract the size of the smaller and equal arrays from K before passing it in. This will continue until either the size of the sub array is 1 or K is in the equal sub array, then the pivot is returned.

```
Illustration 1
        [5, 1, 6, 7, 3, 4, 8]     pivot = 5
           /        |        \
   [1, 3, 4]      [5]     [6, 7, 8]
   smaller       equal     greater
   size 3        size 1    size 3

   K = 3 ≤ 3  →  answer is in smaller


Illustration 2

Call 1: [5, 1, 6, 7, 3, 4, 8]  K=3  pivot=5
        smaller=[1,3,4]  equal=[5]  greater=[6,7,8]
        K ≤ 3 → go to smaller, K stays 3

Call x: **MAGIC**

Final call: [4]  K=1
        size 1 → return 4 ✅
```

Pseudocode

```
Find_Kth_Smallest(arr, k) {

        if arr.length == 1
                return arr[0]

        int pivot = arr[floor(math.rand() * arr.length())]
        smaller = []
        equal = []
        larger = []

        for each x in arr {
                if x > pivot:
                        larger.append(x)
                else if x == pivot:
                        equal.append(x)
                else if x < pivot:
                        smaller.append(x)
        }

        if k <= smaller.length():
                return Find_Kth_Smallest(smaller, k)
        else if k <= smaller.length() + equal.length():
                return pivot;
        else:
                return Find_Kth_Smallest(larger, k-smaller.length()-equal.length())
}
```

Explanation/Justification

Our algorithm is correct because of how we split up the array. Everything in the smaller sub array is less than the pivot, everything in equal is the pivot, and everything in greater is bigger than the pivot. So if the array was sorted smaller would come first then equal then greater. This means if K is less than or equal to the size of smaller the answer has to be in smaller. If K is in the range of the equal array then the answer is the pivot. If K is bigger than both then the answer is in greater, and we subtract the size of smaller and equal from K so it still points to the right number.

We also know it will always finish because the pivot is always taken out of the array before the next call, so the sub array always gets smaller. Eventually the sub array will be size 1 or K will be in the equal array and the pivot gets returned.

Test Cases

Input Array	k	Expected Output	Actual Output	Result
[5, 1, 6, 7, 3, 4, 8]	3	4	4	PASS
[10, 2, 9, 3, 8, 4]	1	2	2	PASS
[10, 2, 9, 3, 8, 4]	6	10	10	PASS
[5, 2, 5, 1, 5, 3]	4	5	5	PASS
[7, 3]	1	3	3	PASS
[-5, 0, -2, 10, 3]	2	-2	-2	PASS
[42, 12, 18, 5, 99]	3	18	18	PASS

Recurrence Relation

The recurrence relation that was derived from our program was as follows:

$$T(n) = T(n-1) + cn$$

The T(n-1) comes from the recursion call in the large if statement at the end of the recursion. We use the -1 because in the worst-case scenario, only one element is removed from the array with each call. The cn added to the recursion comes from the loop through the array that is required to build the smaller/pivot/larger lists. Here's how we solved it:

$$T(n) = T(n-1) + cn$$

$$T(n) = T(n-2) + c(n-1) + cn$$

$$T(n) = T(n-3) + c(n-2) + c(n-1) + cn$$

$$\vdots$$

$$T(n) = T(1) + c(2+3+\dots+n)$$

$$T(n) = c(1) + c\left(\frac{n(n+1)}{2} - 1\right)$$

$$T(n) = \cancel{c} + \frac{cn(n+1)}{2} - \cancel{c}$$

$$T(n) = \frac{cn(n+1)}{2}$$

Benchmarking

n (size)	QuickSelect (s)	Sorting Approach (s)
1,000	0.000204	0.000075
5,000	0.001388	0.000399
10,000	0.002632	0.001043
50,000	0.012062	0.006032
100,000	0.019886	0.011618
1,000,000	0.231215	0.185091

